

UT1: Manejo de Ficheros (Texto, Binarios y XML)

Manual Exhaustivo de Persistencia Física, Flujos I/O, NIO.2 y Serialización en Java

MÓDULO FORMATIVO Acceso a Datos (0486)	UNIDAD DE TRABAJO UT1 / UD01 (30 Horas)	RESULTADOS DE APRENDIZAJE RA1 (CE1.a, CE1.b, CE1.c, CE1.d, CE1.e)
PROFESOR & CURSO Sergio Capdevila Díez · Curso 2026-2027		

Estructura de Contenidos del Documento

1. 1. Introducción Contextualizada y Persistencia

2. 2. Conceptos Fundamentales de I/O y Streams

3. 3. Desarrollo Progresivo y Exhaustivo por Temas

4. 4. Ejemplos Prácticos Graduados (Básico a Avanzado)

5. 5. Código Funcional Paso a Paso: Gestor Empresarial

6. 6. Errores Habituales y Soluciones Técnicas

7. 7. Buenas Prácticas de la Industria por Niveles

8. 8. Matrices Comparativas de Tecnologías

9. 9. Resumen Clave de la Unidad

10. 10. Conceptos que Debes Dominar

11. 11. Glosario y 15 Preguntas de Repaso

1 Introducción Contextualizada y Persistencia

En cualquier sistema informático profesional, los datos almacenados en variables o estructuras en memoria RAM (objetos, arrays, colecciones) tienen un ciclo de vida efímero: desaparecen en cuanto la aplicación finaliza su ejecución, el proceso es detenido o el servidor físico se reinicia. Para conseguir que la información trascienda en el tiempo, es imprescindible implementar mecanismos de **persistencia** en medios no volátiles de almacenamiento secundario (discos de estado sólido SSD, discos duros mecánicos HDD o volúmenes de red NAS/SAN).



Utilidad Real en el Mundo Profesional

Aunque en cursos posteriores y proyectos de gran envergadura la persistencia principal recae sobre Sistemas Gestores de Bases de Datos Relacionales (SGBD) o motores NoSQL, el manejo de ficheros a bajo nivel continúa siendo la columna vertebral técnica de:

- **Registros de auditoría del sistema (Logs):** Generación y rotación de trazas de eventos en servidores web (Apache, Nginx) y microservicios con Logback o Log4j2.
- **Ficheros de configuración de entornos:** Parámetros de arranque del aplicativo en formatos de texto plano (`.properties` , `.env` , `.yaml` , `.json`).
- **Procesos ETL (Extract, Transform, Load) e Ingesta Masiva:** Descarga, análisis y consolidación masiva de millones de registros estructurados en ficheros delimitados (CSV, TSV).
- **Intercambio Seguro B2B y Facturación Electrónica:** Integración entre entidades públicas y bancarias mediante esquemas estándar XML (formato oficial *Facturae*, SEPA ISO 20022).

¿Qué conocimientos previos necesitas? Para seguir con éxito esta unidad formativa debes dominar los fundamentos de Programación en Java de 1º de DAM:

- Programación orientada a objetos: definición de clases, constructores, atributos, visibilidad y métodos.
- Uso de colecciones básicas de la Java Collections Framework: `List` , `ArrayList` y `Map` .
- Gestión estructurada de excepciones mediante bloques `try-catch-finally` y propagación de errores con la cláusula `throws` .

2

Conceptos Fundamentales de I/O y Streams

2.1. La Metáfora Intuitiva del Stream (Flujo de Datos)

Para entender cómo se comunican las aplicaciones con el disco, piensa en una **tubería de agua** que conecta un embalse con un grifo. Tú no abres el grifo y recibes todo el volumen del embalse de golpe en una milésima de segundo; el agua viaja de manera continua, gota a gota o litro a litro, a través de la tubería.

En Java, un **Stream (Flujo)** es exactamente esa tubería: una secuencia ordenada, secuencial y unidireccional de datos que conecta un origen (fuente) con un destino (sumidero). La gran ventaja conceptual del flujo es que ****abstrae por completo el hardware****: para tu código Java, leer 100 bytes de un archivo local en un disco SSD, leerlos desde un puerto de red TCP o recibirlos del teclado utiliza exactamente las mismas llamadas a métodos.

2.2. Las Dos Grandes Jerarquías: Bytes frente a Caracteres

El paquete fundacional de Java (`java.io`) divide todos los flujos en dos árboles de clases completamente separados según el tipo elemental de dato que transportan:

CRITERIO DE CLASIFICACIÓN	FLUJOS DE BYTES (8 BITS / BINARIOS)	FLUJOS DE CARACTERES (16 BITS / UNICODE)
Unidad mínima de datos	byte (8 bits, valores de -128 a 127 / 0 a 255).	char (16 bits, código Unicode UTF-16).
Clase base de Entrada (Lectura)	<code>java.io.InputStream</code>	<code>java.io.Reader</code>
Clase base de Salida (Escritura)	<code>java.io.OutputStream</code>	<code>java.io.Writer</code>
Tipo de contenido idóneo	Imágenes (PNG, JPG), audio, ejecutables, PDFs, ficheros compilados <code>.class</code> .	Texto legible por humanos: archivos <code>.txt</code> , <code>.csv</code> , código fuente, <code>.xml</code> , <code>.json</code> .
Conversión de codificación	No aplica: los bytes se leen o escriben tal cual están físicamente.	Aplica traducción de bytes a caracteres usando un Charset (juego de caracteres).

2.3. Memoria Intermedia (Buffer) y el Patrón Decorador

Acceder a un dispositivo de almacenamiento físico es miles de veces más costoso en tiempo de CPU que operar en la memoria RAM. Si tu programa lee un archivo carácter a carácter invocando un método de disco en cada iteración, la JVM debe solicitar continuas interrupciones al sistema operativo (cambios de contexto de modo usuario a modo kernel), degradando el rendimiento de forma crítica.

💡 El Búfer en Memoria y el Patrón Decorador

Un **Búfer** es un bloque de memoria RAM de alta velocidad (típicamente de 8192 bytes / 8 KB) que actúa como almacén intermedio. Cuando pides el primer carácter, el sistema lee de golpe 8192 caracteres del disco y los guarda en RAM. Las siguientes 8191 lecturas se realizan instantáneamente desde la memoria sin tocar el disco.

Java implementa esto mediante el **Patrón Decorador**: tomas un flujo básico de bajo nivel (por ejemplo `FileReader`) y lo envuelves dentro de una clase decoradora que añade el búfer transparente (`BufferedReader`).

2.4. Gestión de Recursos del Sistema Operativo: try-with-resources

Cada vez que abres un fichero, el núcleo del sistema operativo reserva un **descriptor de fichero (file handle)** en su tabla interna de procesos. Este recurso es limitado por el kernel (a menudo 1024 ficheros abiertos simultáneos en Linux). Si un programa en bucle abre flujos y no los cierra debidamente, se produce una ****fuga de recursos (resource leak)****, bloqueando el archivo en Windows e impidiendo que el servidor abra nuevas conexiones o ficheros con el error fatal:

```
java.io.IOException: Too many open files
```

Para erradicar este problema, desde Java 7 se utiliza la sentencia `try-with-resources`. Cualquier objeto instanciado dentro del paréntesis del `try (...)` que implemente la interfaz `java.lang.AutoCloseable` garantizará la ejecución de su método `close()` de forma automática y ordenada al salir del bloque, incluso si ocurre una excepción grave no controlada.

3 Desarrollo Progresivo y Exhaustivo por Temas

3.1. Ficheros de Texto Plano con la API Clásica (java.io)

Analicemos individualmente las clases fundamentales de la biblioteca clásica `java.io`, sus métodos más usados, ejemplos de código funcional y sus casos de aplicación práctica.

A. Clase `java.io.File`

Concepto: Representación abstracta del nombre y la ruta de un archivo o directorio en el sistema de archivos local. No permite leer ni escribir el contenido del archivo; sirve exclusivamente para consultar metadatos y realizar operaciones sobre el sistema de ficheros.

Instrucciones y Métodos Más Usados:

- `boolean exists()` : Retorna `true` si el archivo o carpeta existe físicamente en el disco.
- `boolean isFile()` y `boolean isDirectory()` : Verifican si el elemento es un archivo regular o una carpeta.
- `String getName()` : Retorna el nombre simple del fichero con su extensión (ejemplo: `"datos.txt"`).
- `String getAbsolutePath()` : Retorna la ruta completa normalizada desde la raíz del sistema.
- `long length()` : Devuelve el tamaño físico del archivo en bytes (para directorios retorna un valor indefinido).
- `boolean createNewFile() throws IOException` : Crea un nuevo archivo vacío de forma atómica; retorna `true` si se creó con éxito.
- `boolean mkdir()` y `boolean mkdirs()` : Crea la carpeta. `mkdirs()` crea además todos los directorios padres intermedios que no existan.
- `boolean delete()` : Borra el archivo o carpeta. Para borrar una carpeta, esta debe estar completamente vacía.
- `File[] listFiles()` : Retorna un array con todos los archivos y subcarpetas que contiene el directorio.

GestionMetadatosFile.java

JAVA 21

```
import java.io.File;
import java.io.IOException;

public class GestionMetadatosFile {
    public static void inspeccionarRuta(String ruta) {
        File f = new File(ruta);

        System.out.println("Ruta analizada: " + f.getAbsolutePath());
        if (f.exists()) {
            System.out.println("¿Es directorio?: " + f.isDirectory());
            System.out.println("¿Es archivo?: " + f.isFile());
            System.out.println("Tamaño físico: " + f.length() + " bytes");
            System.out.println("Permisos - Lectura: " + f.canRead() + " | Escritura: " + f.canWrite());
        } else {
            System.out.println("El elemento no existe en disco.");
        }
    }
}
```

B. Clases `FileReader` y `FileWriter`

Concepto: Flujos de conexión directa con disco físico para leer y escribir caracteres.

Instrucciones y Métodos Más Usados:

- `new FileReader(File archivo, Charset charset)` : Abre el fichero para lectura especificando el juego de caracteres obligatorio.
- `int read()` : Lee un único carácter y lo devuelve como un `int` (su código Unicode de 0 a 65535). Retorna `-1` al alcanzar el final del archivo (EOF).
- `int read(char[] cbuf)` : Lee un bloque de caracteres y los almacena en el array `cbuf`, retornando la cantidad leída.
- `new FileWriter(File archivo, Charset charset, boolean append)` : Abre el archivo para escritura. Si `append` es `true`, añade al final sin borrar el contenido existente.
- `void write(String str)` : Escribe una cadena de texto completa.
- `void flush()` : Fuerza el vaciado inmediato de los datos en disco físico.

C. Clases Decoradoras: BufferedReader y BufferedWriter

Concepto: Envuelven a un `Reader` o `Writer` añadiendo una memoria intermedia de 8 KB. Proporcionan métodos de alto nivel orientados al tratamiento por líneas de texto.

Instrucciones y Métodos Más Usados:

- `String readLine() throws IOException` : Lee una línea completa de texto. Descarta automáticamente los delimitadores de salto de línea (`\r` o `\n`). Devuelve la cadena o `null` si se ha alcanzado el Fin de Fichero (EOF).
- `void newline() throws IOException` : Escribe el salto de línea adecuado según el sistema operativo donde corre el programa (CRLF en Windows, LF en Linux/macOS).

CopiaBufferSegura.java

JAVA 21

```
import java.io.*;
import java.nio.charset.StandardCharsets;

public class CopiaBufferSegura {
    public static void copiarConFiltro(File origen, File destino, String prefijoFiltro) {
        // Bloque try-with-resources: asegura el cierre implícito de ambos flujos
        try (
            BufferedReader br = new BufferedReader(new FileReader(origen, StandardCharsets.UTF_8));
            BufferedWriter bw = new BufferedWriter(new FileWriter(destino, StandardCharsets.UTF_8))
        ) {
            String linea;
            int numLinea = 1;

            // Patrón estándar de lectura línea a línea en Java
            while ((linea = br.readLine()) != null) {
                if (linea.startsWith(prefijoFiltro)) {
                    bw.write "[" + numLinea + " ] " + linea);
                    bw.newLine(); // Salto de línea portable
                }
                numLinea++;
            }
            // Al cerrarse el bloque se ejecuta automáticamente bw.flush() y bw.close()
        } catch (FileNotFoundException e) {
            System.err.println("Error: Archivo de origen no localizable: " + e.getMessage());
        } catch (IOException e) {
            System.err.println("Error crítico de Entrada/Salida: " + e.getMessage());
        }
    }
}
```

Trabajo Síncrono en Aula: Actividad 1.1 y 1.2

Abre tu IDE de desarrollo (Eclipse / IntelliJ) y realiza las **Actividades 1.1 y 1.2 del Cuaderno de Actividades de Aula** para practicar la inspección de archivos con `File` y la lectura por bloques con `BufferedReader`.

3.2. La Revolución de la API Moderna Java NIO.2 (`java.nio.file`)

Introducida en Java 7 y consolidada en las versiones modernas de Java SE, la biblioteca NIO.2 reemplaza los defectos estructurales de `java.io.File` aportando soporte multiplataforma completo, manejo atómico de archivos y programación funcional mediante Streams.

A. Interfaz `java.nio.file.Path` y Métodos Factoría

Concepto: Objeto inmutable que representa la localización de un fichero en el árbol de directorios de manera agnóstica al sistema operativo.

Instrucciones y Métodos Más Usados:

- `Path.of(String primero, String... mas)` : (Recomendado en Java 11+) Construye un `Path` ensamblando las partes sin importar si el SO usa barras inclinadas `/` o invertidas `\`.
- `Path getFileName()` : Retorna el nombre final del recurso.
- `Path getParent()` : Retorna la ruta del directorio contenedor.
- `Path toAbsolutePath()` : Resuelve la ruta relativa respecto al directorio de trabajo actual de la aplicación.
- `Path normalize()` : Elimina redundancias como puntos individuales `.` (directorio actual) y dobles puntos `..` (directorio padre).
- `Path resolve(String otraRuta)` : Concatena dos rutas de forma segura.
- `Path relativize(Path origen)` : Calcula la ruta relativa necesaria para desplazarse desde un `Path` hasta otro.

B. Clase de Utilidad `java.nio.file.Files`

Concepto: Clase estática compuesta enteramente por métodos de utilidad que operan directamente sobre objetos `Path`. A diferencia de `java.io.File` (que devolvía `false` cuando algo fallaba en silencio), `Files` lanza excepciones precisas y descriptivas (`NoSuchFileException`, `DirectoryNotEmptyException`, `FileAlreadyExistsException`).

Operaciones del Sistema de Archivos:

- `boolean exists(Path p)` : Verifica existencia.
- `Path createDirectories(Path p)` : Crea toda la jerarquía de directorios inexistentes.
- `void delete(Path p) throws IOException` : Borra el fichero o lanza excepción si no existe.
- `Path copy(Path origen, Path destino, CopyOption... opciones)` : Copia el fichero. Permite la opción `StandardCopyOption.REPLACE_EXISTING` para sobrescribir.
- `Path move(Path origen, Path destino, CopyOption... opciones)` : Mueve o renombra. Con `StandardCopyOption.ATOMIC_MOVE` asegura que la operación no deje archivos a medio escribir ante caídas de corriente.
- `long size(Path p)` : Retorna el tamaño exacto en bytes.

C. Métodos Atómicos de Lectura y Escritura en Files

Para archivos de tamaño pequeño o moderado (menos de 20-50 MB), `Files` resuelve la lectura o volcado en una única línea de código, abriendo y cerrando los descriptores de forma automática:

- `String readString(Path ruta, Charset cs)` : Lee todo el archivo y lo vuelca directamente en un objeto `String` .
- `Path writeString(Path ruta, CharSequence texto, Charset cs, OpenOption... opciones)` : Escribe el texto en el archivo. Admite opciones de apertura como `StandardOpenOption.CREATE` y `StandardOpenOption.APPEND` .
- `List<String> readAllLines(Path ruta, Charset cs)` : Carga todas las líneas del fichero en una colección `List<String>` en memoria.

D. Procesamiento Funcional de Archivos Gigantes con `Files.lines()`

¿Qué ocurre si el archivo pesa 5 Gigabytes? Usar `readAllLines()` provocaría instantáneamente un error catastrófico `OutOfMemoryError` al intentar almacenar 5 GB de objetos en el Heap de la JVM. Para ficheros masivos se utiliza `Files.lines()` , que genera un flujo perezoso (*Lazy Stream*): las líneas se leen una a una del disco únicamente cuando el pipeline de operaciones funcionales las solicita.

⚠ Regla Crítica: El Stream de `Files.lines()` DEBE cerrarse con `try-with-resources`

A diferencia de los Streams habituales de colecciones (como `lista.stream()`), el `Stream<String>` que produce `Files.lines()` mantiene abierto un descriptor de fichero en el núcleo del sistema operativo. Por tanto, **es obligatorio envolverlo dentro de un `try-with-resources`** para garantizar que el archivo se libere al finalizar la lectura.

AnalisisLogStreamingNIO.java

JAVA 21

```
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.stream.Stream;

public class AnalisisLogStreamingNIO {
    public static void auditarLogServidor(Path rutaLog) {
        // Files.lines abre el canal del SO: DEBE ir en el try()
        try (Stream<String> lineas = Files.lines(rutaLog, StandardCharsets.UTF_8)) {

            long totalErroresCriticos = lineas
                .filter(linea -> linea.contains("[ERROR]") || linea.contains("[FATAL]"))
                .filter(linea -> linea.contains("DatabaseConnectionException"))
                .peek(System.out::println) // Muestra cada línea coincidente
                .count(); // Operación terminal que consume el flujo de forma perezosa

            System.out.println("Total de fallos de base de datos encontrados: " + totalErroresCriticos);

        } catch (IOException e) {
            System.err.println("Fallo al procesar el archivo con NIO.2: " + e.getMessage());
        }
    }
}
```

3.3. Juegos de Caracteres (Charsets) y Trampas de Codificación

Un disco físico no entiende de letras, vocales con tilde ni caracteres especiales; únicamente almacena combinaciones de ceros y unos (bytes). La traducción entre una secuencia binaria y un carácter alfanumérico legible se rige por una tabla de correspondencia matemática denominada **Juego de Caracteres (Charset)**.

- **ASCII (7 bits):** Solo cubre 128 caracteres estándar en inglés (sin tildes, ñ ni símbolos como €).
- **ISO-8859-1 / Windows-1252 (ANSI, 8 bits):** Añade caracteres de idiomas de Europa Occidental, pero asigna códigos incompatibles con otros alfabetos.
- **UTF-8 (Longitud variable, 1 a 4 bytes):** El estándar absoluto de la industria moderna. Representa cualquier carácter del estándar universal Unicode siendo compatible hacia atrás con ASCII. Un carácter común en inglés ocupa 1 byte, pero una 'á' o una 'ñ' ocupan 2 bytes, y el símbolo del euro '€' ocupa 3 bytes.

✗ La Trampa del Charset por Defecto y el Mojibake

Si creas un `new FileReader(archivo)` o invocas `Files.readAllLines(archivo)` sin indicar el juego de caracteres, Java utiliza el valor del sistema anfitrión: `Charset.defaultCharset()`.

- En un equipo de desarrollo Windows en español, el valor por defecto suele ser `windows-1252`.
- En un servidor Linux de despliegue o un contenedor Docker, suele ser `UTF-8`.

Consecuencia: Un archivo con caracteres españoles ("Camión", "Diseño") creado en Windows se leerá en Linux como texto corrupto (*mojibake*: `"CamiÃ±n"` o `"DiseÃ±o"`).

Solución Obligatoria: Declarar siempre la constante `StandardCharsets.UTF_8`.

3.4. Ficheros Binarios y Flujos de Bytes (java.io)

Cuando los datos no representan texto sino estructuras de información binaria (ficheros comprimidos, imágenes o valores numéricos primitivos), se utilizan flujos derivados de `InputStream` y `OutputStream`.

A. Clases FileInputStream y FileOutputStream

Concepto: Conexión byte a byte con el medio de almacenamiento físico.

Instrucciones y Métodos Más Usados:

- `int read()`: Lee el siguiente byte físico (de 0 a 255). Retorna `-1` al alcanzar el final del archivo.
- `int read(byte[] buffer, int off, int len)`: Lee hasta `len` bytes del archivo y los almacena en el array en memoria a partir de la posición `off`. Devuelve el número real de bytes leídos.
- `void write(byte[] b, int off, int len)`: Vuelca el contenido binario del búfer al archivo.

B. Clases DataInputStream y DataOutputStream

Concepto: Decoradores de flujos binarios que permiten leer y escribir variables primitivas de Java (`int`, `double`, `boolean`, `long`) y cadenas de texto en formato independiente del procesador o sistema operativo (codificación estándar Big-Endian de red).

Instrucciones y Métodos Más Usados:

- `writeInt(int v)` (4 bytes) y `int readInt()`: Escritura y lectura de enteros de 32 bits.
- `writeDouble(double v)` (8 bytes) y `double readDouble()`: Números de coma flotante de doble precisión (IEEE 754).
- `writeBoolean(boolean v)` (1 byte) y `boolean readBoolean()`: Valores lógicos.
- `writeUTF(String str)` y `String readUTF()`: Escribe una cadena precedida de 2 bytes que indican la longitud exacta del texto.

```
import java.io.*;

public class RegistroSensoresBinario {
    public static void guardarMedicion(File archivo, int idSensor, double temperatura, boolean alarma) {
        // Abrimos en modo append utilizando FileOutputStream
        try (DataOutputStream dos = new DataOutputStream(
            new BufferedOutputStream(new FileOutputStream(archivo, true)))) {

            dos.writeInt(idSensor);           // Ocupa exactamente 4 bytes
            dos.writeDouble(temperatura);     // Ocupa exactamente 8 bytes
            dos.writeBoolean(alarma);         // Ocupa exactamente 1 byte
            dos.writeUTF("CALIBRADO");        // 2 bytes de longitud + caracteres

        } catch (IOException e) {
            System.err.println("Error al guardar datos binarios: " + e.getMessage());
        }
    }
}
```

3.5. Ficheros de Acceso Aleatorio (RandomAccessFile)

En todos los flujos vistos hasta ahora (secuenciales), para leer el dato número 1.000 era obligatorio leer primero los 999 datos anteriores. `java.io.RandomAccessFile` rompe esta limitación: se comporta como un gran array de bytes almacenado en disco donde puedes saltar de forma instantánea a cualquier índice mediante su **Puntero de Fichero (File Pointer)**.

A. Conceptos y Modos de Apertura

- Modo `"r"` : Apertura exclusiva para lectura. Cualquier intento de escritura provocará un error `IOException`.
- Modo `"rw"` : Apertura simultánea para lectura y escritura. Si el fichero no existe en disco, lo crea automáticamente.

B. Métodos Fundamentales de Posicionamiento

- `void seek(long posicionByte) throws IOException` : Sitúa el puntero de lectura/escritura exactamente en el byte número `posicionByte`. El primer byte del archivo es la posición 0.
- `long getFilePointer() throws IOException` : Consulta en qué índice de byte exacto se encuentra el puntero en este momento.
- `long length() throws IOException` : Devuelve el tamaño total en bytes del fichero.
- `void setLength(long nuevaLongitud)` : Trunca o agranda el archivo al tamaño indicado.

C. La Necesidad de Registros de Longitud Fija

Para que sea posible saltar directamente al registro número N sin leer el resto, **cada registro del archivo debe ocupar exactamente el mismo número de bytes** en el disco físico.

La fórmula matemática universal para posicionarse en el registro N (siendo el primer registro el número 1) es:

$$\text{Posición en bytes (Offset)} = (N - 1) \times \text{Tamaño del Registro en bytes}$$

⚠ La Trampa de las Cadenas en RandomAccessFile

Si escribes un String con `writeUTF()`, los bytes que ocupa variarán según el nombre: "Ana" ocupará 5 bytes (2 de cabecera + 3 caracteres) mientras que "Alejandro" ocupará 11 bytes. Esta variación rompe el tamaño fijo del registro y provoca que la fórmula del `seek()` aterrice en bytes erróneos corruptos.

Solución Obligatoria: Fijar siempre una longitud constante en caracteres para cada campo de texto (por ejemplo 30 caracteres = 60 bytes en Unicode), rellenando con espacios en blanco los sobrantes.

GestorEmpleadosRAF.java

JAVA 21

```
import java.io.File;
import java.io.IOException;
import java.io.RandomAccessFile;

public class GestorEmpleadosRAF {
    // Definición de la estructura de registro fija:
    // ID (int = 4 bytes) + Apellido (20 chars = 40 bytes) + Salario (double = 8 bytes) = 52 bytes
    public static final int TAM_REGISTRO = 52;
    public static final int LONG_MAX_APELLIDO = 20;

    public static void modificarSalario(File archivo, int numRegistro, double nuevoSalario) {
        try (RandomAccessFile raf = new RandomAccessFile(archivo, "rw")) {
            long totalRegistros = raf.length() / TAM_REGISTRO;
            if (numRegistro < 1 || numRegistro > totalRegistros) {
                System.out.println("Registro fuera de rango.");
                return;
            }

            // Calculamos el offset exacto: (N-1) * 52 bytes + 44 bytes para ir directo al salario
            long offsetSalario = (long)(numRegistro - 1) * TAM_REGISTRO + 4 + (LONG_MAX_APELLIDO * 2);
            raf.seek(offsetSalario); // Saltamos directamente al salario en disco
            raf.writeDouble(nuevoSalario); // Sobreescribimos in-situ los 8 bytes

            System.out.println("Salario actualizado con éxito en el registro " + numRegistro);
        } catch (IOException e) {
            System.err.println("Error de acceso aleatorio: " + e.getMessage());
        }
    }
}
```

3.6. Serialización y Deserialización de Objetos en Java

La **Serialización** es el mecanismo nativo de Java que permite tomar un objeto vivo en la memoria RAM y convertir toda su estructura interna, atributos y objetos anidados en una secuencia de bytes que puede persistirse en disco o enviarse por la red. La **Deserialización** es el proceso inverso: reconstruir el objeto original en memoria a partir de los bytes guardados.

A. La Interfaz Marcadora Serializable y el Identificador serialVersionUID

Para que una clase pueda ser serializada debe implementar la interfaz `java.io.Serializable`. Esta interfaz es "marcadora" (no tiene métodos que implementar), pero indica a la JVM que permite su inspección reflectiva.

El papel crítico de `serialVersionUID`: Es una constante numérica de 64 bits (tipo `long`) que identifica unívocamente la versión estructural de la clase:

```
private static final long serialVersionUID = 1L;
```

Si tú no defines esta constante explícitamente, la Máquina Virtual de Java calculará un hash automático en base a los atributos y métodos de la clase. Si añades un método o compilas con una versión diferente de JDK, el hash cambiará y el programa fallará con una excepción `InvalidClassException`, impidiéndote recuperar los datos guardados.

B. El Modificador `transient`

Si un atributo de tu clase está marcado con la palabra clave `transient`, el motor de serialización de Java lo omitirá por completo. Es obligatorio para:

- Datos confidenciales o sensibles: contraseñas en texto plano, tarjetas de crédito.
- Recursos dependientes del entorno: descriptores de ficheros, sockets de red, hilos `Thread` o conexiones JDBC activas.
- Campos calculables que pueden regenerarse a partir de otros atributos.

C. El Problema de la Doble Cabecera en modo Append

Cuando creas una instancia de `ObjectOutputStream`, este escribe obligatoriamente en el archivo una **cabecera mágica de 4 bytes** (`0xAC 0xED 0x00 0x05`).

Si abres un fichero existente en modo append para añadir nuevos objetos:

```
new ObjectOutputStream(new FileOutputStream("datos.dat", true)) // ¡PELIGRO!
```

Se escribirá una segunda cabecera mágica en mitad del archivo. Al intentar leerlo posteriormente con `ObjectInputStream`, al alcanzar ese punto la JVM esperará datos de un objeto y encontrará una cabecera, colapsando inmediatamente con la excepción:

```
java.io.StreamCorruptedException: invalid type code: AC
```

Solución Profesional de la Industria: Crear una subclase especializada de `ObjectOutputStream` que anule la escritura de la cabecera cuando el archivo ya contenga datos.

MiObjectOutputStreamAppend.java

JAVA 21

```
import java.io.IOException;
import java.io.ObjectOutputStream;
import java.io.OutputStream;

public class MiObjectOutputStreamAppend extends ObjectOutputStream {
    public MiObjectOutputStreamAppend(OutputStream out) throws IOException {
        super(out);
    }

    // Sobreescribimos el método para no emitir la cabecera si estamos añadiendo datos
    @Override
    protected void writeStreamHeader() throws IOException {
        // No invocar super.writeStreamHeader() para evitar la corrupción del archivo
        reset();
    }
}
```

3.7. Persistencia e Intercambio en Formato XML

El lenguaje **XML (eXtensible Markup Language)** es un estándar universal estructurado, legible por humanos y máquinas, ampliamente utilizado en el intercambio de información empresarial. En Java coexisten tres tecnologías fundamentales para su tratamiento:

A. Procesamiento en Memoria con DOM (Document Object Model)

Concepto: Carga el documento XML completo y construye un árbol jerárquico de nodos en la memoria RAM. Permite navegar hacia adelante, hacia atrás y modificar cualquier nodo en tiempo real.

Clases e Instrucciones Más Usadas:

- `DocumentBuilderFactory` y `DocumentBuilder` : Fábricas para construir el parser.
- `Document parse(File f)` : Parsea el archivo y retorna el objeto `Document` raíz.
- `NodeList getElementsByTagName(String etiqueta)` : Busca todos los elementos coincidentes en el árbol.
- `Element createElement(String etiqueta)` : Crea un nuevo nodo en memoria.
- `void appendChild(Node nuevo)` : Añade el nuevo nodo como hijo en la jerarquía.
- `TransformerFactory` y `Transformer` : Vuelca el árbol DOM modificado de vuelta al archivo XML en disco con sangrado legible (`OutputKeys.INDENT`).

B. Procesamiento Reactivo Orientado a Eventos con SAX (Simple API for XML)

Concepto: Parser unidireccional que lee el archivo XML de principio a fin sin cargar nada en la memoria RAM. A medida que encuentra una etiqueta de apertura, un texto o una etiqueta de cierre, dispara un evento que es atendido por los métodos de una clase manejadora (`DefaultHandler`).

Métodos del Manejador (`DefaultHandler`):

- `startDocument()` y `endDocument()` : Invocados al comenzar y finalizar el archivo.
- `startElement(String uri, String localName, String qName, Attributes atts)` : Se dispara al encontrar una etiqueta de apertura (por ejemplo `<empleado id="101">`).
- `characters(char[] ch, int start, int length)` : Se dispara cuando el parser lee el texto contenido dentro de la etiqueta.
- `endElement(String uri, String localName, String qName)` : Se dispara al encontrar la etiqueta de cierre (`</empleado>`).

C. Mapeo Declarativo con JAXB (Java Architecture for XML Binding)

Concepto: Permite vincular directamente clases Java (POJOs) con estructuras de etiquetas XML mediante anotaciones, eliminando la necesidad de programar manualmente recorridos por árboles o bucles de nodos.

- `@XmlRootElement(name = "catalogo")` : Marca la clase que representa el elemento raíz del XML.
- `@XmlElement(name = "articulo")` : Asocia un campo con una etiqueta hija.
- `@XmlAttribute(name = "id")` : Guarda el atributo como parte de la etiqueta XML.
- **Marshalling**: Objeto Java $\rightarrow$ Documento XML (usando `Marshaller.marshal()`).
- **Unmarshalling**: Documento XML $\rightarrow$ Objeto Java (usando `Unmarshaller.unmarshal()`).

4 Ejercicios y Ejemplos Prácticos Graduados

Nivel 1 (Básico): Contador Estadístico con BufferedReader

Permite asimilar el control del Fin de Fichero (EOF) y la manipulación de cadenas de texto línea por línea.

ContadorPalabras.java

JAVA 21

```
import java.io.*;
import java.nio.charset.StandardCharsets;

public class ContadorPalabras {
    public static void analizarFichero(File archivo) {
        int totalLineas = 0;
        int totalPalabras = 0;

        try (BufferedReader br = new BufferedReader(new FileReader(archivo, StandardCharsets.UTF_8))) {
            String linea;
            while ((linea = br.readLine()) != null) {
                totalLineas++;
                String linealimpia = linea.trim();
                if (!linealimpia.isEmpty()) {
                    String[] palabras = linealimpia.split("\\s+");
                    totalPalabras += palabras.length;
                }
            }
            System.out.println("Líneas leídas: " + totalLineas + " | Total palabras: " + totalPalabras);
        } catch (IOException e) {
            System.err.println("Fallo al leer: " + e.getMessage());
        }
    }
}
```

Nivel 2 (Intermedio): Transformador y Validador CSV con NIO.2

Combina `Files.lines()`, filtrado de expresiones regulares y escritura atómica en fichero de salida.

```
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.*;
import java.util.List;
import java.util.stream.Stream;

public class ProcesadorTransaccionesCSV {
    public static void filtrarVentasAltas(Path csvOrigen, Path txtDestino, double importeMinimo) {
        try (Stream<String> lineas = Files.lines(csvOrigen, StandardCharsets.UTF_8)) {

            List<String> aprobadas = lineas
                .skip(1) // Omitimos la cabecera CSV
                .filter(l -> {
                    String[] c = l.split(";");
                    return c.length >= 4 && Double.parseDouble(c[3]) >= importeMinimo;
                })
                .map(l -> {
                    String[] c = l.split(";");
                    return String.format("REF:%s | CLIENTE:%s | TOTAL:%.2f EUR", c[0], c[2], Double.parseDouble(c[3]));
                })
                .toList();

            Files.write(txtDestino, aprobadas, StandardCharsets.UTF_8,
                StandardOpenOption.CREATE, StandardOpenOption.TRUNCATE_EXISTING);

            System.out.println("Transacciones procesadas y guardadas: " + aprobadas.size());
        } catch (IOException e) {
            System.err.println("Error procesando CSV: " + e.getMessage());
        }
    }
}
```

Nivel 3 (Avanzado / Profesional): Actualizador Transaccional con RandomAccessFile

Modifica directamente el registro de un producto en disco sin recorrer el archivo ni crear copias temporales.

```
import java.io.File;
import java.io.IOException;
import java.io.RandomAccessFile;

public class ActualizadorStockRAF {
    // Registro: ID (int, 4 bytes) + Nombre (30 chars, 60 bytes) + Stock (int, 4 bytes) = 68 bytes
    public static final int TAM_REGISTRO = 68;

    public static boolean actualizarStock(File bdArticulos, int idProducto, int unidadesRestar) {
        try (RandomAccessFile raf = new RandomAccessFile(bdArticulos, "rw")) {
            // El ID determina la posición física exacta del registro: (id - 1) * 68
            long posRegistro = (long)(idProducto - 1) * TAM_REGISTRO;
            if (posRegistro + TAM_REGISTRO > raf.length()) {
                return false; // El producto no existe en el archivo
            }

            // Saltamos directamente a la posición del campo Stock (offset + 64 bytes)
            raf.seek(posRegistro + 64);
            int stockActual = raf.readInt();

            if (stockActual >= unidadesRestar) {
                raf.seek(posRegistro + 64); // Reposicionamos para sobrescribir
                raf.writeInt(stockActual - unidadesRestar);
                return true;
            }
            return false; // Stock insuficiente
        } catch (IOException e) {
            System.err.println("Error crítico en acceso directo: " + e.getMessage());
            return false;
        }
    }
}
```

5 Código Funcional Paso a Paso: Gestor Empresarial

La siguiente clase compila en un único componente profesional las mejores prácticas de la industria: resolución segura de rutas relativas con NIO.2, creación controlada de directorios padre, sanitización de codificaciones y lectura bufferizada con

`try-with-resources`.

FileUtilsPro.java

JAVA 21

```
import java.io.*;
import java.nio.charset.StandardCharsets;
import java.nio.file.*;
import java.util.List;

public class FileUtilsPro {

    /**
     * Escribe un contenido textual de forma segura en disco, garantizando la existencia
     * de los directorios padre y aplicando codificación estandarizada UTF-8.
     */
    public static void guardarTextoSeguro(Path rutaFichero, String contenido) throws IOException {

        // 1. Asegurar que las carpetas contenedoras existan
        Path directorioPadre = rutaFichero.getParent();
        if (directorioPadre != null && !Files.exists(directorioPadre)) {
            Files.createDirectories(directorioPadre);
        }

        // 2. Escritura atómica con NIO.2 aplicando UTF-8
        Files.writeString(
            rutaFichero,
            contenido,
            StandardCharsets.UTF_8,
            StandardOpenOption.CREATE,
            StandardOpenOption.TRUNCATE_EXISTING,
            StandardOpenOption.WRITE
        );
    }

    /**
     * Copia un archivo de un directorio a otro aplicando reemplazo si ya existía.
     */
    public static void respaldarFichero(Path origen, Path destino) throws IOException {
        if (!Files.exists(origen)) {
            throw new NoSuchFileException("El archivo de origen no existe: " + origen);
        }
        Files.copy(origen, destino, StandardCopyOption.REPLACE_EXISTING, StandardCopyOption.COPY_ATTRIBUTES);
    }
}
```

✗ Error 1: Fugas de Descriptores de Archivo del Sistema Operativo

¿Qué ocurre? Abrir flujos con `new FileReader()` o `new FileInputStream()` y cerrarlos manualmente al final del método con `fr.close()`.

Causa técnica profunda: Si se produce cualquier excepción antes de llegar a la línea del `close()`, el manejador queda retenido en la tabla de descriptores del kernel. En Windows el archivo queda bloqueado (imposibilitando que otro proceso lo borre) y en Linux colapsa el servidor al alcanzar el límite del sistema (*Too many open files*).

Solución profesional: Emplear siempre `try-with-resources`.

✗ Error 2: Corrupción de Índices en RandomAccessFile por Cadenas Dinámicas

¿Qué ocurre? Escribir cadenas mediante `writeUTF()` en un fichero de acceso directo asumiendo un tamaño de registro constante.

Causa técnica profunda: `writeUTF()` escribe 2 bytes de cabecera más los bytes reales del texto. Al cambiar el tamaño del String entre un empleado y otro, la fórmula matemática $(N-1) \times \text{tam}$ hace aterrizar el puntero en mitad de un dato numérico, corrompiendo la lectura.

Solución profesional: Asignar un número fijo de caracteres y rellenar los sobrantes con espacios en blanco.

✗ Error 3: El Desastre de la Doble Cabecera en la Serialización (modo append)

¿Qué ocurre? Usar `new ObjectOutputStream(new FileOutputStream(file, true))` para añadir objetos al final de un archivo serializado existente.

Causa técnica profunda: Cada constructor de `ObjectOutputStream` inyecta una cabecera de 4 bytes (`0xAC0xED0x000x05`). Al leer el segundo bloque, la JVM espera un objeto y se topa con otra cabecera mágica, lanzando `StreamCorruptedException`.

Solución profesional: Crear una subclase que anule el método `writeStreamHeader()` en modo append.

✗ Error 4: Colapso OutOfMemoryError al procesar XML Masivos con DOM

¿Qué ocurre? Parsear un documento XML de 300 MB mediante `DocumentBuilder.parse()`.

Causa técnica profunda: DOM instancia en la memoria RAM objetos Java para cada elemento, atributo, espacio en blanco y fragmento de texto. Un XML de 300 MB consume entre 1.2 y 1.8 GB de memoria Heap, colapsando la máquina virtual.

Solución profesional: Utilizar un parser reactivo orientado a eventos como **SAX** o **StAX**.

7 Buenas Prácticas de la Industria por Niveles

NIVEL DE CUMPLIMIENTO	REQUISITO TÉCNICO	JUSTIFICACIÓN PROFESIONAL
Obligatorio (Producción)	Especificar siempre <code>StandardCharsets.UTF_8</code> .	Evita el mojibake al migrar código entre entornos Windows y contenedores Linux.
Obligatorio (Producción)	Envolver todo flujo I/O en <code>try-with-resources</code> .	Garantiza la liberación inmediata de manejadores en el sistema operativo.
Obligatorio (Producción)	Usar <code>Path.of()</code> en vez de cadenas con barras directas.	Previene errores de resolución de rutas entre sistemas operativos distintos.
Altamente Recomendado	Definir siempre <code>private static final long serialVersionUID</code> .	Evita que modificaciones en la clase impidan deserializar datos previamente guardados.
Altamente Recomendado	Bufferizar flujos de entrada y salida con clases envoltorio.	Reduce drásticamente el número de operaciones I/O en disco físico acelerando el sistema.
Estándar de Arquitectura	No bloquear el hilo de ejecución principal durante I/O intensivo.	Mantiene la reactividad de la interfaz y la concurrencia de peticiones en servidores web.

8 Matrices Comparativas de Tecnologías

Tabla A: `java.io.File` frente a `java.nio.file.Path`

CARACTERÍSTICA	API CLÁSICA (<code>java.io.File</code>)	API MODERNA (<code>java.nio.file.Path & Files</code>)
Año de introducción	Java 1.0 (Año 1996)	Java 7 (NIO.2, Año 2011) / Java 11
Gestión de errores	Retorna <code>false</code> si falla (silencioso).	Lanza excepciones ricas (<code>NoSuchFileException</code> , etc.).
Enlaces simbólicos y metadatos POSIX	Soporte nulo o extremadamente precario.	Soporte completo multiplataforma (permisos, ACLs, atributos).
Operaciones atómicas	No admitidas por el API.	Admitidas con <code>StandardCopyOption.ATOMIC_MOVE</code> .
Integración con Programación Funcional	Nula.	Nativa mediante <code>Files.lines()</code> , <code>Files.walk()</code> .

Tabla B: Comparativa de Tecnologías de Persistencia en Ficheros

TECNOLOGÍA	TIPO DE FICHERO	VELOCIDAD DE ACCESO	LEGIBILIDAD	CASO DE USO IDÓNEO
Texto con NIO.2 / Buffer	Texto plano (UTF-8)	Media / Alta	Excelente (humana)	Logs, archivos de configuración, CSVs.
DataStream / Output	Binario secuencial	Muy Alta	Nula (binaria)	Series de datos numéricos y sensores.
RandomAccessFile	Binario indexado	Instantánea por ID ($O(1)$)	Nula (binaria)	Mini-motores de bases de datos, catálogos fijos.
Serialización de Objetos	Binario nativo Java	Alta	Nula (binaria)	Cachés en disco, persistencia de grafos complejos.
XML (DOM / SAX / JAXB)	Texto jerárquico	Media / Baja	Excelente (humana)	Factura electrónica, interoperabilidad entre lenguajes.

9 Resumen Clave de la Unidad

- **Streams:** Abstracción de tubería unidireccional. Se dividen en flujos de *Bytes* (`InputStream` / `OutputStream`) y flujos de *Caracteres* (`Reader` / `Writer`).
- **Buffer:** Imprescindible para el rendimiento; almacena 8 KB intermedios en RAM evitando peticiones microscópicas al disco.
- **NIO.2:** Representa el estándar moderno basado en `Path.of()` y métodos atómicos de la clase `Files` . Para ficheros gigantes, `Files.lines()` procesa de forma perezosa mediante Streams.
- **StandardCharsets.UTF_8:** Parámetro ineludible en toda lectura o escritura de texto para evitar la corrupción de tildes o caracteres multilingües (*mojibake*).
- **RandomAccessFile:** Permite leer y escribir en cualquier posición arbitraria mediante `seek()` ; requiere obligatoriamente que los registros tengan una longitud constante en bytes.
- **Serialización:** Almacena objetos en disco con `ObjectOutputStream` . Requiere implementar `Serializable` , declarar `serialVersionUID` y usar `transient` para omitir datos sensibles. En modo append exige una subclase que anule `writeStreamHeader()` .
- **XML:** DOM construye el árbol en memoria RAM (ideal para edición de ficheros pequeños/medianos); SAX procesa eventos al vuelo con consumo cero de memoria (ideal para ficheros gigantes); JAXB mapea automáticamente clases POJO a etiquetas XML.

10 Conceptos que Debes Dominar

- ✓ **Diferenciar conceptualmente** cuándo utilizar flujos de bytes frente a flujos de caracteres.
- ✓ **Justificar técnicamente** la necesidad de envolver flujos en buffers y usar `try-with-resources` .
- ✓ **Manejar con soltura** la API moderna Java NIO.2: creación de rutas con `Path.of()` , operaciones con `Files` y lectura perezosa con `Files.lines()` .
- ✓ **Construir programas robustos** que no dependan del Charset del sistema anfitrión, forzando `StandardCharsets.UTF_8` .
- ✓ **Calcular con precisión matemática** la posición en bytes de un registro en un `RandomAccessFile` y actualizarlo in-situ.
- ✓ **Implementar clases serializables** declarando explícitamente su `serialVersionUID` y aislando datos sensibles con `transient` .
- ✓ **Diagnosticar y resolver** el fallo de doble cabecera (`StreamCorruptedException`) al añadir objetos serializados en modo append.
- ✓ **Seleccionar el analizador XML adecuado** (DOM frente a SAX o JAXB) justificando la decisión en base a los requisitos de memoria y volumen de datos del sistema.

11 Glosario Técnico y Banco de Preguntas de Repaso

Glosario de Términos Fundamentales

- **Stream (Flujo):** Canal de comunicación unidireccional y secuencial que transporta bytes o caracteres entre un origen y un destino.
- **Búfer:** Área temporal de memoria RAM de alta velocidad diseñada para agrupar operaciones de lectura/escritura y reducir los accesos a disco físico.
- **Descriptor de Fichero (File Handle):** Identificador numérico asignado por el núcleo del sistema operativo a cada archivo abierto por un proceso.
- **Charset:** Mapa de correspondencia estandarizado entre secuencias binarias de bytes y caracteres legibles (alfanuméricos o gráficos).
- **Mojibake:** Texto corrupto e ilegible resultante de decodificar una secuencia de bytes con un Charset diferente al que se utilizó para escribirlo.
- **Evaluación Perezosa (Lazy Evaluation):** Estrategia de procesamiento donde los elementos de un flujo no se cargan en memoria hasta que la operación terminal los solicita.
- **Puntero de Fichero (File Pointer):** Cursor numérico que señala el índice exacto de byte en el que tendrá lugar la siguiente lectura o escritura en un `RandomAccessFile`.
- **serialVersionUID:** Identificador de versión de 64 bits que asegura que una clase Java coincide estructuralmente con los datos serializados que se intentan recuperar.
- **transient:** Modificador reservado en Java que indica al motor de serialización que ignore dicho atributo al guardar el objeto.
- **DOM (Document Object Model):** Especificación de parseo XML que crea en memoria RAM un árbol de objetos navegable bidireccionalmente.
- **SAX (Simple API for XML):** Parser unidireccional basado en eventos que lee XML continuo con un consumo mínimo y constante de memoria RAM.
- **Marshalling / Unmarshalling:** Proceso de transformar un grafo de objetos Java en un documento XML (Marshalling) y el proceso inverso (Unmarshalling) mediante JAXB.

Banco de 15 Preguntas de Repaso (Sin Respuesta Inmediata)

Utiliza estas 15 preguntas abiertas para comprobar tu nivel de comprensión técnica antes de enfrentarte a las pruebas de evaluación y exámenes del módulo:

Pregunta 1: Explica detalladamente la diferencia arquitectónica entre la jerarquía de clases que descende de `InputStream` / `OutputStream` y la que descende de `Reader` / `Writer`. ¿En qué situación técnica cometerías un error grave si utilizas un `Reader` para copiar una imagen JPEG?

Pregunta 2: ¿Qué ventaja aporta envolver un `FileReader` en un `BufferedReader`? Explica qué ocurre a nivel de llamadas al sistema operativo y qué método de alto nivel añade para facilitar la lectura.

Pregunta 3: Un programador junior afirma: "Siempre utilizo `Files.readALLLines()` porque es más cómodo y escribe menos código". ¿Bajo qué circunstancias de producción esta afirmación provocará la

caída total del servidor de aplicaciones? ¿Qué método alternativo de NIO.2 debes utilizar en su lugar y por qué?

Pregunta 4: ¿Por qué es estrictamente obligatorio cerrar el flujo generado por `Files.lines()` mediante un bloque `try-with-resources`, mientras que un Stream generado a partir de un `ArrayList` no lo requiere?

Pregunta 5: Explica qué es el *mojibake*. Pon un ejemplo de qué ocurre con la letra 'ñ' si un archivo guardado con codificación Windows-1252 es abierto por un programa Java que utiliza UTF-8 por defecto. ¿Cómo se soluciona en el código?

Pregunta 6: Menciona tres diferencias fundamentales entre la antigua clase `java.io.File` y la interfaz moderna `java.nio.file.Path` combinada con `java.nio.file.Files`.

Pregunta 7: En un fichero de acceso aleatorio gestionado con `RandomAccessFile`, si cada registro ocupa exactamente 68 bytes, escribe la instrucción exacta en Java para posicionar el puntero en el inicio del registro número 12.

Pregunta 8: ¿Por qué es indispensable que los registros de un `RandomAccessFile` tengan una longitud fija en bytes? ¿Qué problema ocurriría si guardas cadenas de texto usando `writeUTF()` sin normalizar su tamaño?

Pregunta 9: Si modificas los 8 bytes de un salario en la mitad de un archivo mediante `RandomAccessFile.seek()` y `writeDouble()`, ¿se pierden o desplazan los datos que vienen después en el fichero? Justifica cómo gestiona el disco esta operación.

Pregunta 10: ¿Para qué sirve el identificador `serialVersionUID` en una clase que implementa `Serializable`? ¿Qué ocurre si no lo declaras de forma explícita y posteriormente agregas un método a la clase?

Pregunta 11: ¿Qué utilidad tiene la palabra clave `transient`? Pon dos ejemplos concretos de atributos de una clase en un sistema empresarial que deban ser declarados obligatoriamente como `transient`.

Pregunta 12: Explica la causa física a nivel de bytes del error `java.io.StreamCorruptedException: invalid type code: AC` al añadir objetos con `ObjectOutputStream` en modo append. ¿Cómo se soluciona técnicamente?

Pregunta 13: Te encargan desarrollar un microservicio para procesar catálogos XML de proveedores que pesan más de 800 Megabytes cada uno. ¿Qué tecnología de procesamiento XML elegirías (DOM o SAX) y por qué motivo técnico de uso de memoria?

Pregunta 14: En el procesamiento con SAX, ¿cuándo se ejecutan los métodos `startElement()`, `characters()` y `endElement()` del `DefaultHandler` ?

Pregunta 15: ¿Qué diferencias existen entre los procesos de *Marshalling* y *Unmarshalling* en la biblioteca JAXB y qué anotación es indispensable en la clase raíz?